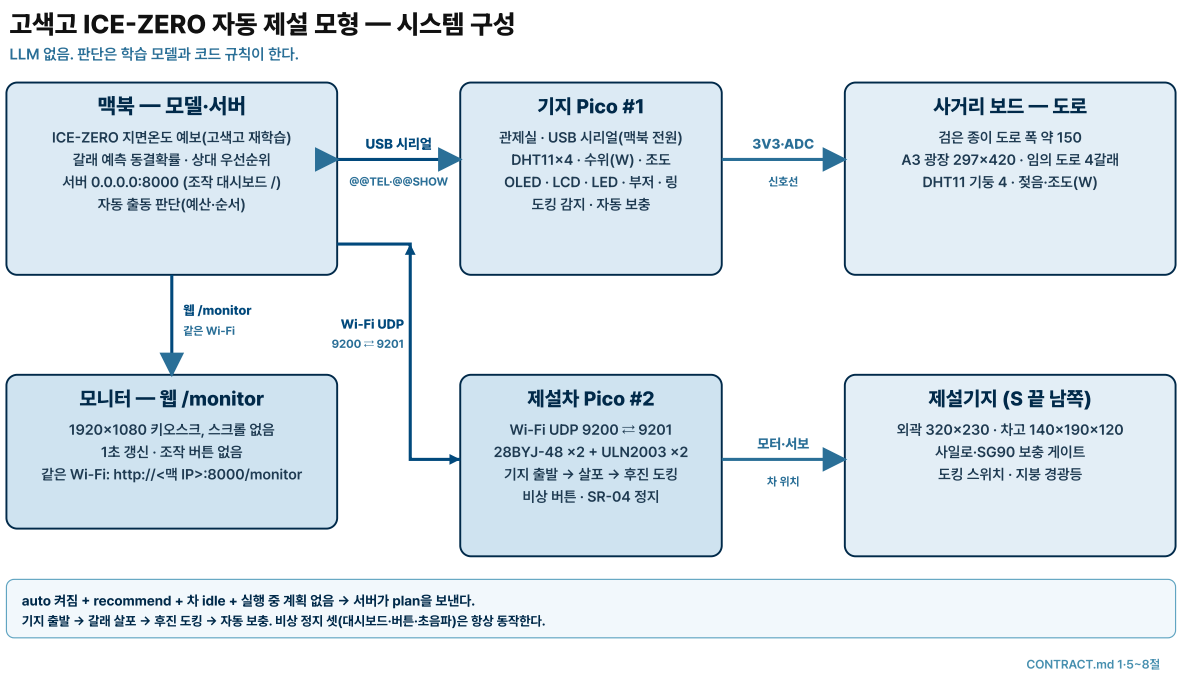
라즈베리파이 시스템 설계도

고색고 ICE-ZERO 자동 제설 모형 · 기지 Pico #1(USB) + 제설차 Pico #2(Wi-Fi) + 맥북 서버 · 2026-10-31 12:08 KST



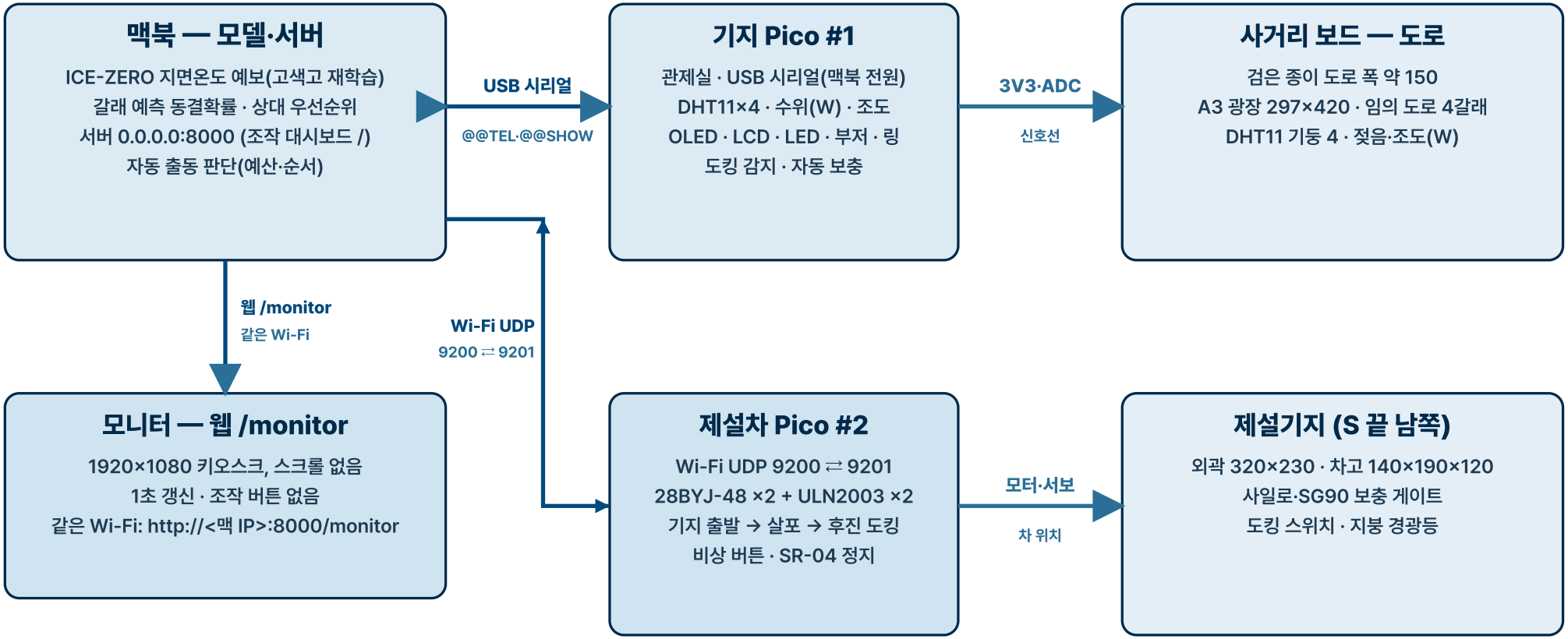
차례

1. 1. 구성
2. 2. 데이터 흐름
3. 3. Pico 두 대의 역할
4. 4. 핀표
5. 5. 전원
6. 6. 프로토콜
7. 7. 상태기계
8. 8. 안전
9. 9. 보정 방법
10. 10. 보드 배치
11. 부록: 모니터 화면 · 참고 지도

기준 문서는 CONTRACT.md다. 이 PDF는 docs/SYSTEM_DESIGN.md를 그대로 옮겨 만든 것이다. 그림 원본은 docs/img/*.svg다. 도로는 임의 도로(모형)이고, 보드 치수는 사진 추정값이라 실측해서 config에 넣는다. 실물 배선·주행은 아직 검증하지 않았다.

고색고 ICE-ZERO 자동 제설 모형 — 시스템 구성

LLM 없음. 판단은 학습 모델과 코드 규칙이 한다.



auto 켜짐 + recommend + 차 idle + 실행 중 계획 없음 → 서버가 plan을 보낸다.
기지 출발 → 갈래 살포 → 후진 도킹 → 자동 보충. 비상 정지 셋(대시보드·버튼·초음파)은 항상 동작한다.

CONTRACT.md 1·5~8절

docs/img/system.svg

1. 구성

부분	하는 일	실행 위치
모델·엔진	수원 ASOS(지점 119)로 다시 학습한 지면온도 예보, 갈래별 예측 동결확률·우선순위	맥북 `model/icezero/`
서버·대시보드	상태 API, 센서 융합, 자동 출동 판단, 이벤트 기록	맥북 `server/gosaek_server.py`
기지 Pico #1	갈래 온습도·젖음·조도 측정, 기지 전광판·경광등, 도킹 감지·자동 보충	`pico/sensors/`
제설차 Pico #2	스텝모터 주행, 서보 살포, 초음파 정지, UDP 통신	`pico/car/`
사거리 보드	검은 종이 도로 + A3 광장, 센서 기둥, 남쪽 제설기지	책상

- 맥북은 모델을 돌리고 판단한다. **LLM은 쓰지 않는다.** 화면 문구도 '예측 동결확률', '상대 우선순위'다.
- Pico 두 대는 서로 직접 말하지 않는다. 모두 맥북을 거친다(센서는 USB, 차는 Wi-Fi UDP).
- 모니터는 같은 서버의 `/monitor`를 전체 화면으로 띄운다(계약 5절).

2. 데이터 흐름

1. **학습(한 번)**: `data/raw/119/` ASOS 원본 → 피처 → XGBoost 회귀·분류 → `model/models/119/*.json`.
2. **도로(한 번)**: Overpass로 고색고 반경 1.5km 도로·교량·건물 → `data/roads/cross_arms.json`.
3. **실행(매 순간)**: 서버가 1초마다 엔진 `step()`을 부른다.
    - 엔진은 그 시각의 예보와 갈래 속성으로 `p_freeze` · `level` · `priority`를 낸다.
    - 서버는 기지 Pico의 `@@TEL`을 받아 갈래 `dt`(현장 온도차)와 W 갈래 젖음을 `local`로 넣는다.
    - `recommend`가 참이고 차가 idle이면 `plan`을 보낸다.
4. **표시**: 서버가 `@@SHOW`로 기지 Pico의 링·OLED·LCD에 갈래 상태를 보내고(`beacon`이면 경광등), 대시보드·모니터에 같은 내용을 그린다.
5. **복귀**: 차가 `arm_done`을 보내면 서버가 그 갈래를 3시간 동안 처리됨(`done`)으로 둔다.

3. Pico 두 대의 역할

기지 Pico #1 (USB 시리얼, 관제실)

- DHT11 4개로 갈래별 온·습도를 재고, 수위 센서로 W 갈래 젖음을, 조도 센서로 W 그늘 밝기를 읽는다.
- 2초마다 `@@TEL` 한 줄을 보낸다.

- 맥이 죽어도 로컬 경고는 계속 돈다: 어느 갈래든 `t ≤ WARN_T`이고 `rh ≥ WARN_RH`이거나, 젖었고 W 갈래 `t ≤ WARN_T`면 3색 LED·부저를 켠다. 3회 연속 거짓이면 끈다.
- 차가 도킹하면(스위치 1초 유지) 사일로 게이트를 800ms 열어 호퍼를 채운다. 한 번 도킹에 한 번만, 20초 쿨다운.
- DHT11은 0~50°C만 잰다. 영하 실외에서는 값이 없거나 0에 붙으므로, 야외 설치 때는 DS18B20 같은 다른 센서가 필요하다(계약 밖 선택 품목).

제설차 Pico #2 (Wi-Fi UDP)

- `plan`을 받으면 갈래 순서대로 ① 방위 회전 ② 360mm 전진(중심 110~340mm에서만 서보 열기) ③ 180° 회전 ④ 중심 복귀를 한다.
- 500ms마다 `tel`로 상태·위치·진행 단계를 보낸다.
- 명령 `stop` · `home`과 차 버튼, 초음파 장애물 정지는 항상 우선한다.

4. 핀표

4.1 기지 Pico #1 — `pico/sensors/config.py`

장치	핀	계약
DHT11 N / E / S / W	GP2 / GP3 / GP4 / GP5	6절
수위감지 AO(젖음, W)	GP26 (ADC0)	6절
조도 AO(W 그늘)	GP27 (ADC1)	6절
OLED SSD1306 128×32 I2C0	SDA GP0, SCL GP1, 0×3C	6절
3색 LED	R GP10, G GP11, B GP12	6절
능동 부저	GP13	6절
도킹 스위치(차고 뒤벽)	GP14 (풀업, 눌리면 0)	6절
보충 게이트 SG90	GP15 (닫힘 0°, 열림 60°)	6절
WS2812 링 DIN(경광등)	GP16	6절
I2C 1602 LCD(전광판)	같은 I2C0, 0×27	6절

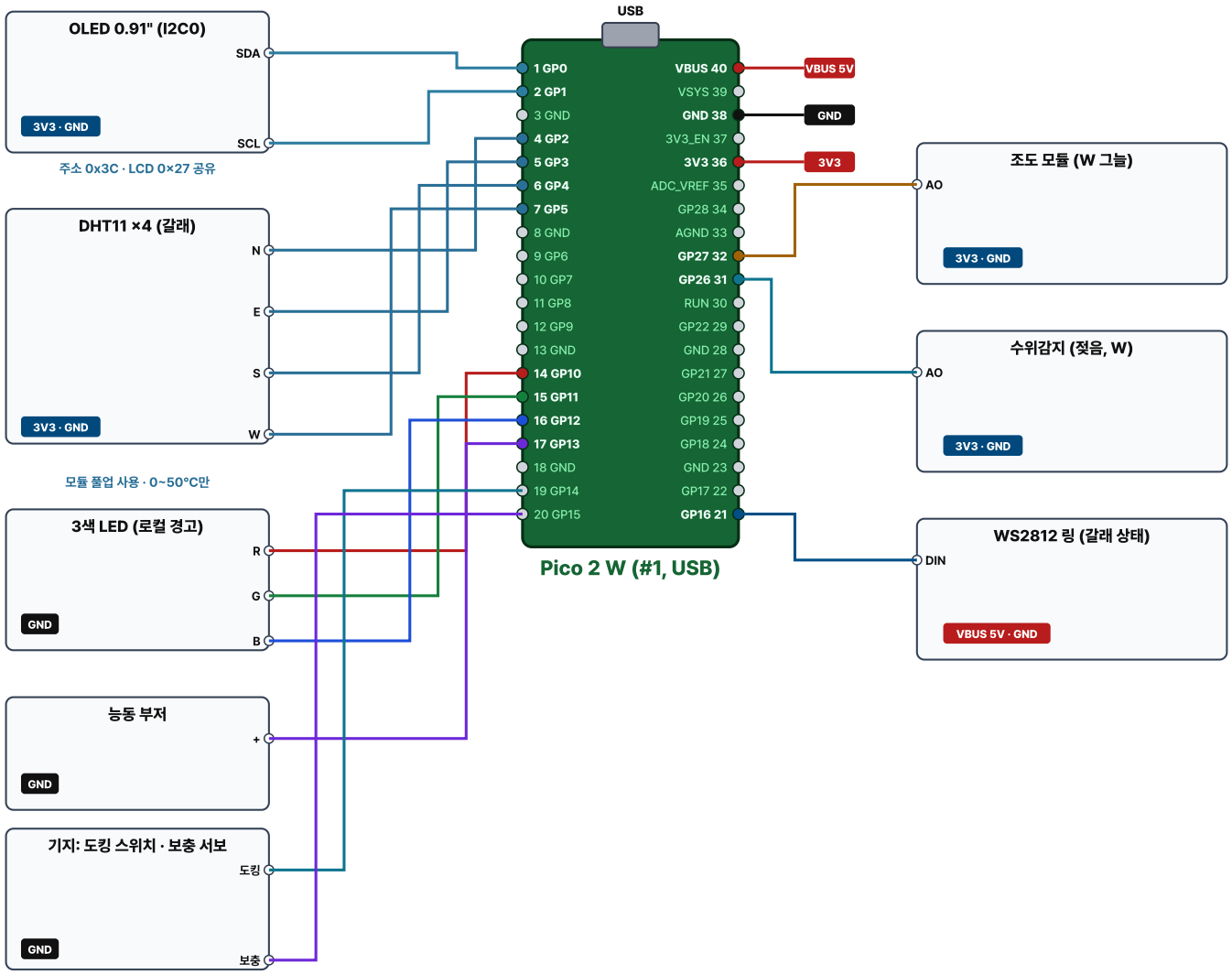
4.2 제설차 Pico #2 — `pico/car/config.py`

장치	핀	계약
왼쪽 28BYJ-48 (ULN2003 IN1~IN4)	GP2, GP3, GP4, GP5	7절
오른쪽 28BYJ-48 (ULN2003 IN1~IN4)	GP6, GP7, GP8, GP9	7절
SG90 살포 서보	GP15 (닫힘 0°, 열림 70°)	7절
SR-04 TRIG / ECHO	GP16 / GP17	7절
택트 스위치(정지·재개, 풀업)	GP14	7절
3색 LED	R GP18, G GP19, B GP20	7절
능동 부저	GP21	7절

기구 상수: `WHEEL_D_MM=65`, `TRACK_MM=105`, `STEPS_PER_REV=4096` (하프스텝), `STEP_US=1200`. 두 모터를 마주 보게 달기 때문에 오른쪽 모터 방향을 소프트웨어에서 반전한다.

센서 배선

센서 Pico #1 배선 — DHT11×4 · 수위 · 조도 · OLED · 3색 LED · 부저 · 링



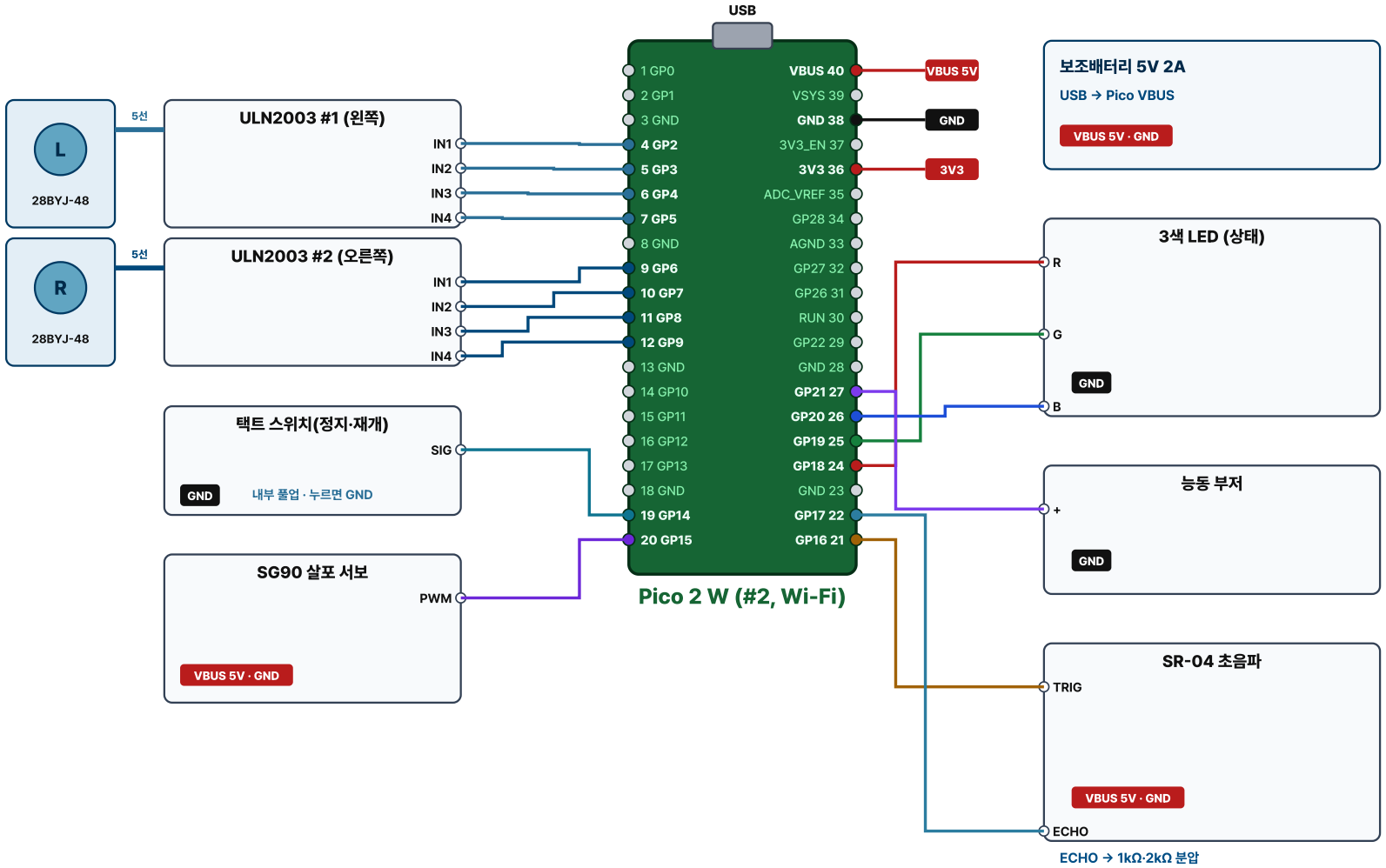
모든 모듈 VCC는 3V3, GND는 공통이다. 링만 VBUS 5V를 쓰고 DIN에 330Ω을 넣는다.

DHT11 모듈은 풀업이 붙어 있어 데이터선만 이으면 된다. 수위·조도 AO는 3V3 기준 0~65535.

docs/img/sensor_wiring.svg

차 배선

제설차 Pico #2 배선 — 스텝모터 ×2 · 서보 · 초음파 · 버튼 · LED · 부저



ULN2003 VCC는 VBUS, GND는 Pico와 공통. stop·버튼·장애물이면 ULN 입력을 모두 LOW로 내려 모터를 세운다.

SR-04 ECHO는 5V이므로 1kΩ·2kΩ 분압으로 3.3V에 맞춘다. 스텝모터 두 개를 마주 보게 달아 오른쪽 방향을 반전한다.

docs/img/car_wiring.svg

5. 전원

부하	전압	전류(대략)	전원
Pico #2 자체	5V→3.3V	60~120mA	보조배터리 USB
28BYJ-48 두 개	5V (ULN2003 VCC=VBUS)	코일 2상 × 2대 ≈ 400~600mA	보조배터리
SG90 서보	5V	100~250mA(이동 순간)	보조배터리
SR-04	5V	15mA	보조배터리
WS2812 링(센서 쪽)	5V (VBUS)	밝기 48에서 최대 100mA	맥북 USB

- 5V 2A 보조배터리면 위 합(순간 1A 안팎)을 넣기 충분하다. 스텝모터를 돌릴 때 서보를 동시에 움직이지 않는다(펌웨어가 순서를 지킨다).
- ULN2003 VCC는 **VBUS**, GND는 Pico와 공통으로 묶는다. 모터 전원을 Pico 3V3에서 끌어오지 않는다.
- 맥북 USB는 기지 Pico #1에만 쓴다. 5V 부하는 링·LCD·서보 하나라 USB 전류로 충분하다.
- 서보·모터를 한꺼번에 움직이면 전압이 떨어져 Pico가 재부팅할 수 있다. 그래서 `STEP_US` (스텝 간격)를 넉넉히 잡아 속도와 순간 전류를 낮춘다.
- SR-04 ECHO는 5V 출력이다. 1kΩ·2kΩ 분압으로 3.3V에 맞춘다(계약 권장).

6. 프로토콜

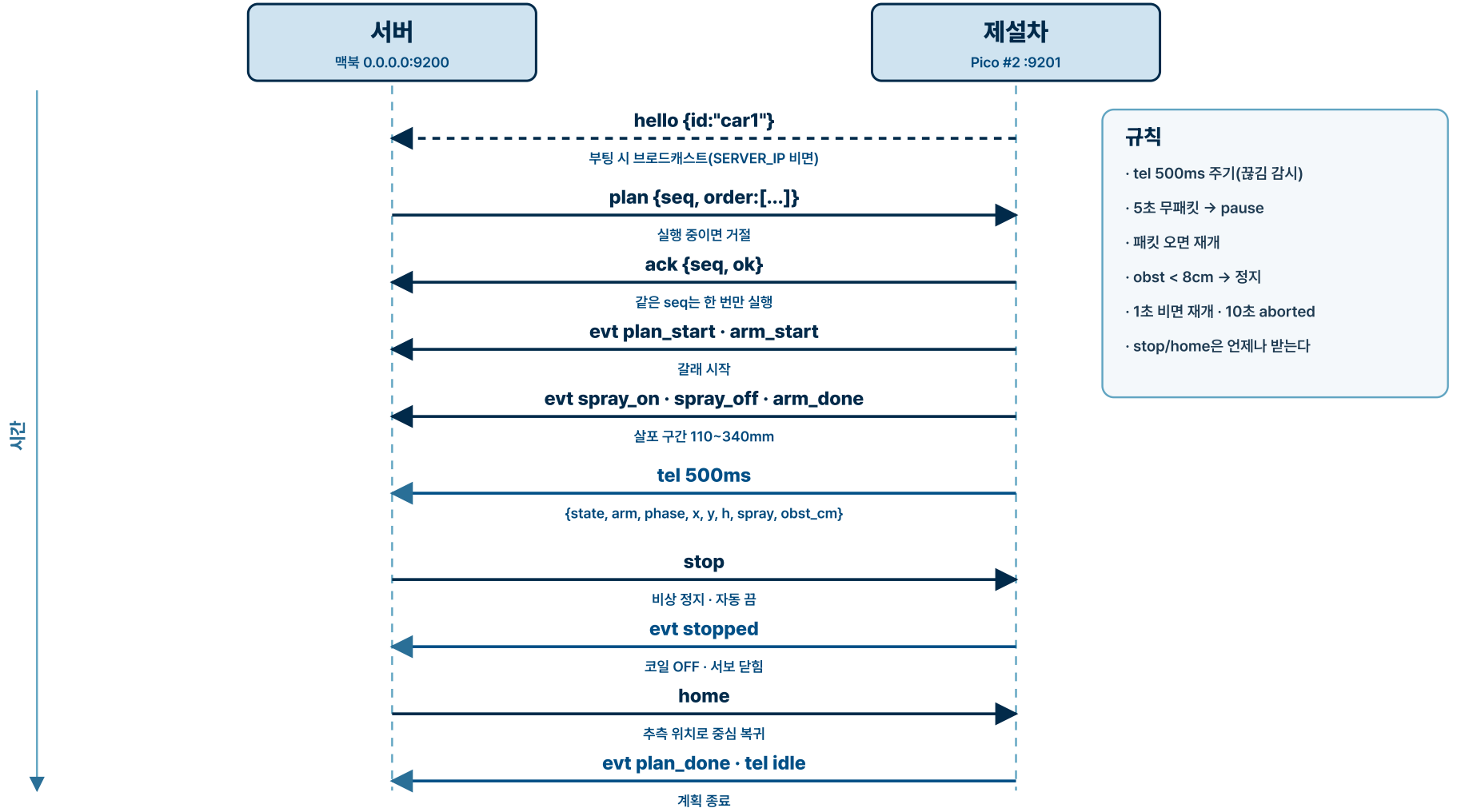
6.1 맥 ↔ 기지 Pico (USB 시리얼, 115200, 한 줄 = 한 메시지)

- Pico → 맥, 2초마다: `@@TEL {"t":{...},"rh":{...},"ok":{...},"wet_raw":40000,"wet":false,"light_raw":21000,"warn":false,"dock":false,"refills":0,"refilling":false,"up":12345}`
- 맥 → Pico: `@@SHOW {"arms":{"N":"risk1","E":"risk3"},"msg":"AUTO ON E>W","beacon":true}` — `msg` 는 16자 이내 ASCII(OLED 글꼴이 한글을 그리지 못함). `beacon` 이 참이면 링이 경광등처럼 돈다.
- 맥 → Pico: `@@REFILL {"ms":800}` — 수동 보충. 도킹 상태에서만 연다.

6.2 맥 ↔ 제설차 (Wi-Fi UDP, 서버 9200 수신, 차 9201 수신) — 프로토콜

UDP 프로토콜 — 서버(맥북) ⇄ 제설차 Pico #2

한 줄 = 한 데이터그램(JSON). 서버 9200 수신, 차 9201 수신.



ack는 받은 seq를 되돌려 준다. 서버는 ack ok=false면 계획을 접고 이벤트에 남긴다.

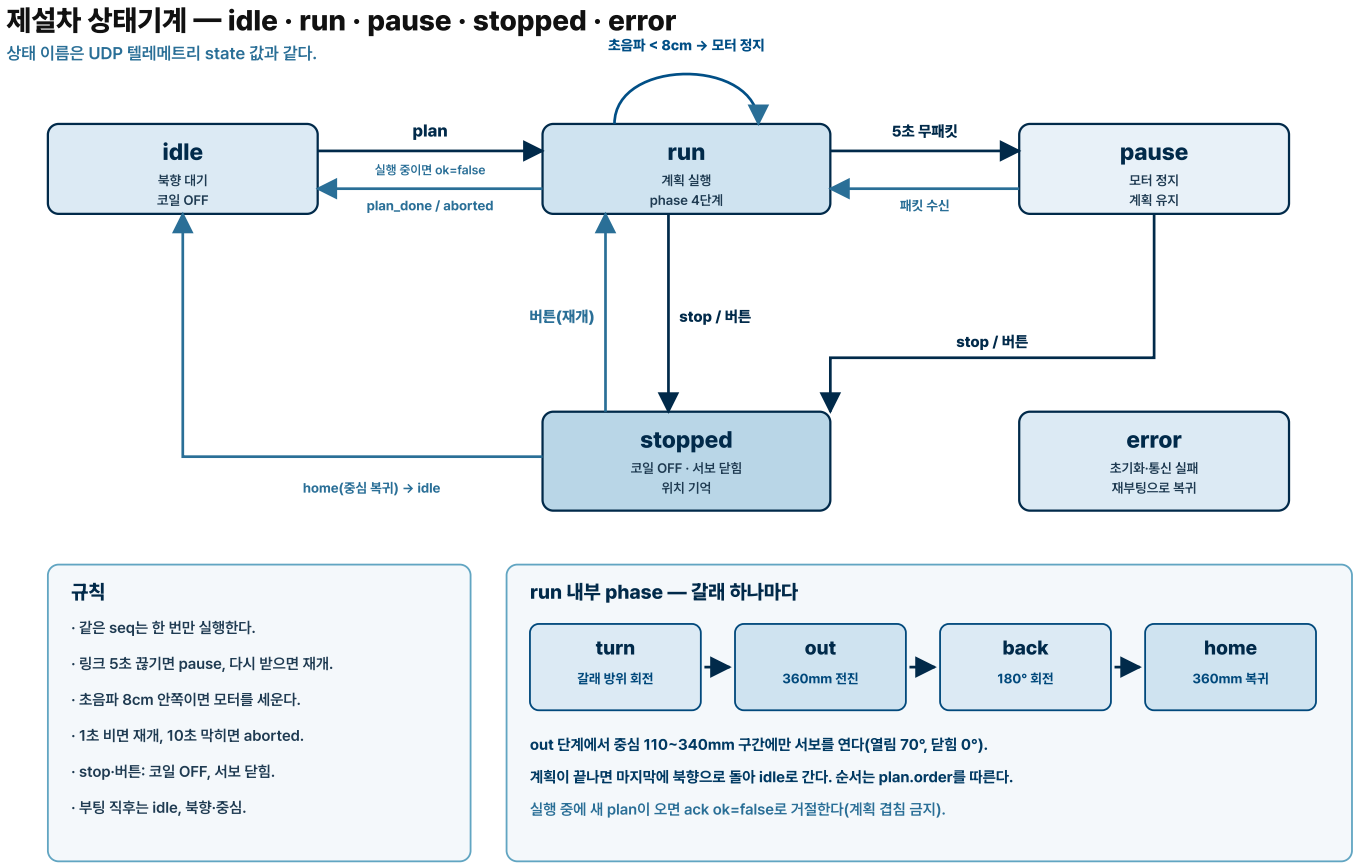
차가 없거나 링크가 끊기면 서버는 '모의 실행'으로 갈래당 3초 뒤 완료 처리한다.

docs/img/protocol.svg

방향	메시지
차 → 서버	`hello {id}` (부팅, `SERVER_IP`가 비면 브로드캐스트)
서버 → 차	`plan {seq, order}` / `stop` / `home` / `ping`
차 → 서버	`ack {seq, ok, reason}`
차 → 서버	`evt {e, seq, arm}` — `plan_start` · `arm_start` · `spray_on` · `spray_off` · `arm_done` · `plan_done` · `stopped` · `obstacle` · `resumed` · `aborted`
차 → 서버	`tel` 500ms — `state`, `arm`, `phase`, `x`, `y`, `h`, `spray`, `obst_cm`, `up`

- 같은 `seq`는 한 번만 실행한다(재전송 대비).
- `LINK_TIMEOUT_MS=5000` 넘게 서버 패킷이 없으면 `pause`, 다시 받으면 `resume`.

7. 상태기계 — 제설차 상태



- `idle` → `plan` → `run` (depart → turn → out → back … → home) → `plan_done` → `idle` .
- **depart**: 도킹 위치에서 북쪽으로 `BASE_DIST_MM` 전진해 중심에 닿는다. order에 S가 있으면 이 길에서 S 살포 구간(중심까지 100~390)을 처리하고 S의 `arm_start` · `arm_done` 을 보낸다.
- **turn → out → back**: 갈래 방위로 회전 → `ARM_OUT_MM[arm]` 전진(나가는 길에만 살포) → 180° 회전 → 같은 거리 전진해 중심 복귀.
- **home**: 북향으로 돌아 `BASE_DIST_MM + DOCK_OVERDRIVE_MM` 만큼 **후진**해 차고에 들어간다. 끝나면 추측 자세를 도킹 위치·북향으로 다시 맞춘다.
- `run` 중 5초 무패킷이면 `pause` , 패킷이 오면 `run` 으로 돌아간다.
- `stop` ·버튼·장애물 → 모터 코일 OFF, 서보 닫힘. 버튼을 다시 누르면 남은 구간을 재개한다.
- `home` 명령을 받으면 지금 위치에서 중심으로 돌아온 뒤 위 3번(후진 도킹)을 한다.
- 초음파 8cm 안쪽이면 모터를 세우고, 1초 비면 재개, 10초 넘게 막히면 `aborted` 를 보낸다. **후진 중에는 앞쪽 초음파를 무시한다**(차고 벽을 장애물로 보지 않게).

docs/img/car_states.svg

8. 안전

- 자동 출동에도 **비상 정지 세 갈래**가 항상 살아 있다: 대시보드 `stop`, 차 버튼, 초음파 장애물 정지.
- `stop`은 자동 모드를 끈다. 다시 켜야 자동 출동이 재개된다.
- 도킹 스위치는 차가 뒤벽을 미는 힘으로 눌린다. 1초 이상 계속 눌려야 보충하고, 한 번 도킹에 한 번만 연다(쿨다운 20초).
- 차가 없거나 링크가 끊기면 서버는 모의 실행(갈래당 3초)으로 표시만 한다. 실제 살포로 오해하지 않도록 대시보드에 표시한다.
- 결빙을 '확정'했다고 쓰지 않는다. 화면·로그 모두 '예측 동결확률', '상대 우선순위'다.
- 모형의 살포물은 마른 모래·스티로폼 알갱이다. 실제 제설제를 쓰지 않는다(전자부·우드락 손상).

9. 보정 방법

9.1 기구 상수(차)

1. 바퀴 지름을 캘리퍼로 잰다(예: 65.3mm). `WHEEL_D_MM`에 넣는다.
2. 두 바퀴 중심 사이 거리를 잰다(예: 104mm). `TRACK_MM`에 넣는다.
3. **1m 직진 시험**: 바닥에 1m 줄을 긋고 북향에서 출발시킨다. 실제 이동 거리 `d`를 재서 `STEPS_PER_REV = 현재값 × 1000 / d`로 고친다(한 바퀴 스텝은 하프스텝 4096이 기본).
4. **360° 회전 시험**: 바닥에 테이프로 기준선을 긋고 제자리 360° 회전시킨다. 한 바퀴보다 덜 돌면 `TRACK_MM`을 키우고, 더 돌면 줄인다. 좌우가 어긋나면 오른쪽 모터 스텝 보정값을 조정한다.
5. 두 시험을 번갈아 두 번 하면 1cm 안쪽으로 맞는다.

9.2 젖음 센서

1. 마른 채로 `@@TEL`의 `wet_raw`를 읽는다(예: 61000).
2. 물을 묻혀 다시 읽는다(예: 18000).
3. 중간값을 `WET_THRESHOLD`에 넣고, 젖을 때 값이 작아지면 `WET_WHEN_LOW=True`.

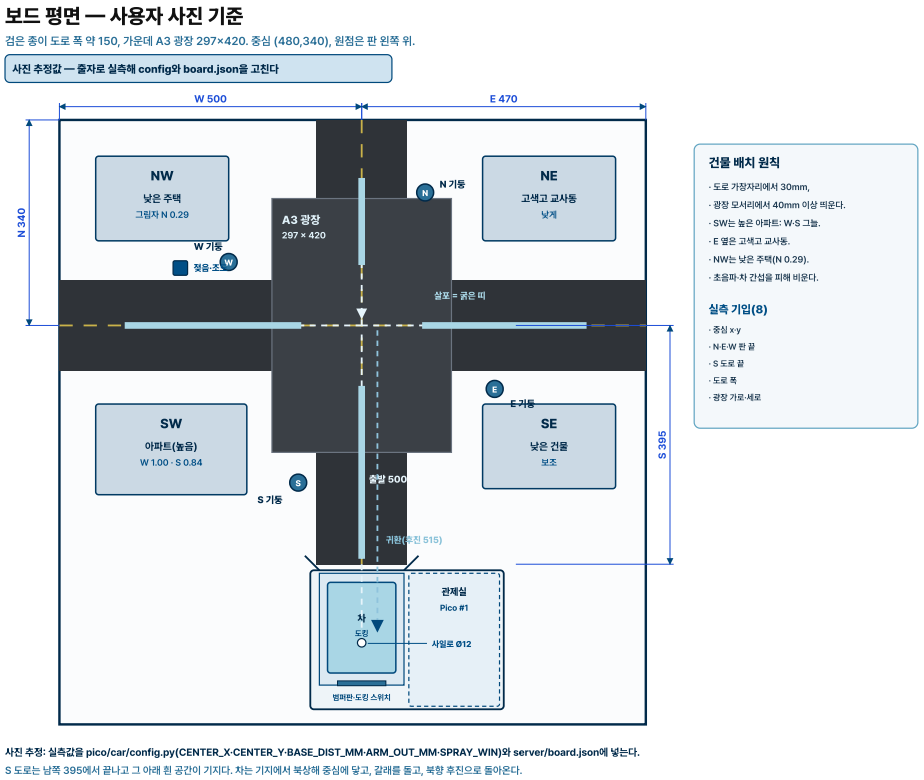
9.3 현장 온도 보정

- 서버는 시작 30초 평균(또는 `/api/sensors/baseline` 시점)을 갈래 기준값으로 잡는다.
- 이후 `dt = T_k − baseline_k`를 엔진 `local`로 넣는다. 실내 시연에서는 기준값을 다시 잡아야 한다.
- 센서가 10초 넘게 조용하면 서버는 `local`을 비운다(모델만으로 판단).

9.4 모델

- 학습·검증 수치와 방법 선택은 `docs/MODEL.md`를 본다.
- 재학습: `python3 -m model.icezero.train --stn 119 --force`.

10. 보드 배치 — 보드 배치 — 사진 추정



- 흰 우드락 위 검은 종이 도로. 도로 폭 약 150, 가운데 A3 광장 297(동서)×420(남북).
- 중심 (CENTER_X, CENTER_Y) = 사진 추정 (480, 340). 원점은 판 왼쪽 위, x는 동쪽·y는 남쪽.
- 판 끝까지(중심 기준): N 340 · E 470 · W 500. S 도로는 395에서 끝나고 그 아래가 기지다.
- 센서 기둥(DHT11, 중심 기준): N (+105,−220) · E (+220,+105) · S (−105,+260) · W (−220,−105).
- 젖음·조도 센서는 W 갈래(−300,−95) 길가.
- 차 홈 = 기지 도킹 위치 (CENTER_X, CENTER_Y + BASE_DIST_MM) , 북향. 방위 N=0° E=90° S=180° W=270°(시계 방향).
- 차가 멈추는 거리: N 250 · E 380 · W 400. 서보는 중심 100부터 그 거리까지 연다.
- 건물은 네 모퉁이에만, 도로 가장자리 30 · 광장 모서리 40 이상 띄운다. SW는 높은 아파트(W·S 그늘), E 옆은 고색고 교사동, NW는 낮은 주택.
- 모든 값은 **사진 추정**이다. 실측해 pico/car/config.py 와 server/board.json 에 넣는다(BASE_AND_CAR.md 1절).

docs/img/board_v2.svg

차 배치

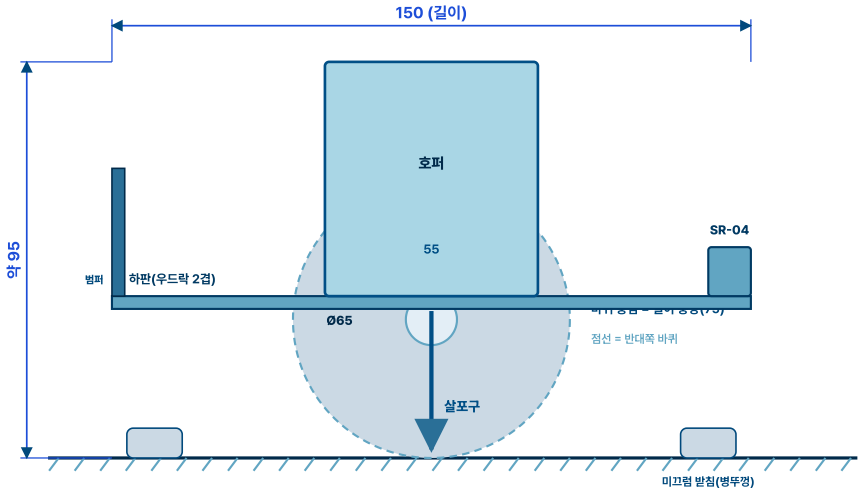
제설차 제작 도면 — 하판 100×150, 전체 113×150×약 95

단위 mm, 화면 4px = 1mm. 28BYJ-48 두 개를 마주 보게 달고 축은 바깥으로 낸다. 앞이 위(북).

(a) 위에서 — 앞이 위(북)



(b) 옆에서 — 앞이 오른쪽



(c) 조립 순서

1. 하판 100×150 재단(2겹)
2. 모터 브래킷 고정
3. 바퀴 3겹 원판 접착
4. D축 구멍 3×5 뚫기
5. 바퀴·고무줄 장착
6. ULN2003 ×2 부착
7. Pico·배터리 부착
8. 호퍼 접착·구멍
9. SG90 패들 장착
10. SR-04 앞면 장착
11. LED·버튼·부저·범퍼
12. 배선·보정 시험

바퀴: 우드락 3겹 원판 Ø65 + 고무줄 타이어 + D축 구멍 3×5. 앞뒤 미끄럼 받침은 병뚜껑을 쓴다.

무게 배치: 뒤쪽 배터리·Pico, 가운데 호퍼, 앞쪽 ULN·SR-04. 바퀴 접지를 살리려 무게 중심을 조금 뒤에 둔다.

호퍼 입구는 뒤 범퍼에서 75mm다. 기지 사일로 출구와 같은 자리라 도킹하면 그대로 떨어진다.

- 하판 100(폭)×150(길이), 바퀴 포함 전체 113×150, 높이 약 95. 바퀴 Ø65×8, 두 바퀴 중심 간격 105.
- 28BYJ-48 두 개를 마주 보게 달아 축을 바깥으로 내고, 앞뒤는 병뚜껑 미끄럼 받침으로 받친다.
- 뒤쪽에 배터리·Pico, 가운데 호퍼 50×50×55(살포구는 차체 아래), 앞쪽에 ULN2003·SR-04, 뒤면에 범퍼판 60×30.
- 제작·재단·조립 12단계는 `BASE_AND_CAR.md` 4절.

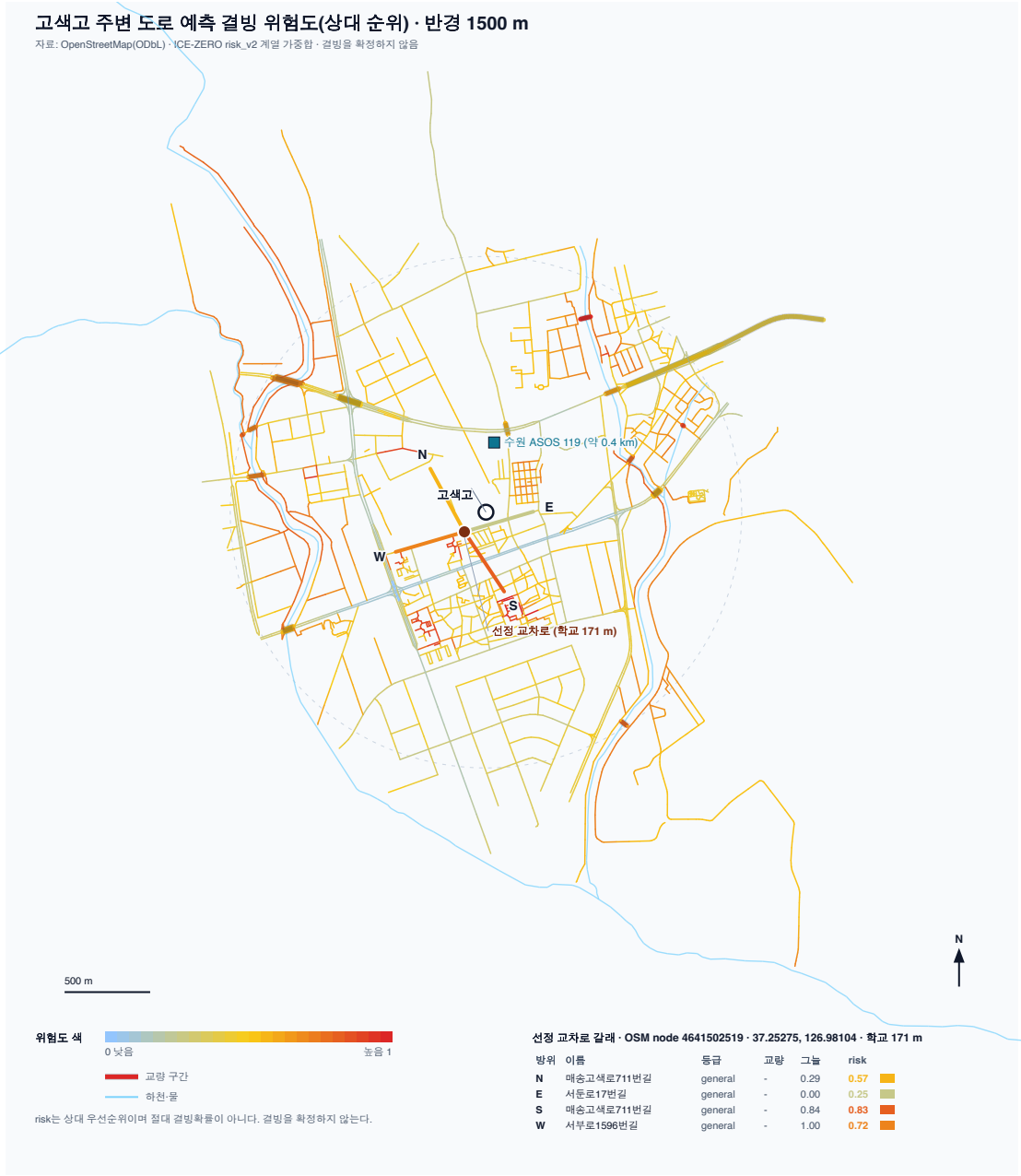
docs/img/car_build.svg

부록 A. 모니터 화면(/monitor, 1920×1080)



실제 엔진 + 가짜 차·기지로 캡처

부록 B. 참고: 고색고 주변 실제 도로 위험도(모형 도로와 별개)



OSM 기반 상대 우선순위 · 모형 갈래는 임의 도로(cross_arms.json)